

Furkan Büyüksarıkulak

ID 22002097

EE-102, Section 003

Lab 2 : Introduction to VHDL

Experiment 2

04.10.2023

AIM:

The purpose of this experiment is learn how to using Vivado and code in VHDL. In addition, it was aimed to design combinational circuits. Then, to learn the basics of FPGA programming by compiling the VHDL code with Basys3 FPGA board.

DESIGN SPECIFICATIONS:

In this experiment, I referenced the health status of a pepper plant to simulate a real-life example. To illustrate that in combinational circuits I have designed this circuit with four inputs and one output as noted below.

Sunlight	: input
Water	: input
Poison	: input
Aphid	: input
Health status of pepper plant : output	

It was associated with the AND gate with inputs 'sunlight' and 'water', which yielded the result 'M'. Then inputs of 'poison' and 'aphid' are related with NOR gate which gives the result 'N'. The results, Gates 'M' and 'N', are associated with an AND gate that gives the 'health' as a output. The scheme of the design is seen in Figure1. This design is intended to Show the factors that affect the health status of the pepper plant and the relationship between the factors that affect whether it is healthy or unhealthy.

METHODOLOGY:

Initially, in this experiment we design a circuit about real-life situation by using Vivado in VHDL. My real-life situation is about the health status of a pepper plant ; which consists of four external factors as follows : sunlight, water, poison and, aphid. As a result of the effects of these four factors, the health status of the pepper plant can be positive or negative. The condition for the pepper plant to have a positive health condition is that it has sunlight and water and, at the same time it should not contain any poison or aphids. Namely, if it is deprived of either water or sunlight , and likewise if any poison or aphid is present, the health of the plant will be impaired. After I decided on this real-life example, I used AND and NOR Gates to recreate the relationship between the inputs. To creation of the combinational circuit shown in Figure 1. I synthesized the module. We design a testbench code using the 'port map' and 'wait for' functions, and our goal here is to simulate the results we have. The simulation we run is as shown in Figure 2. Then, we designed a code that allows us to associate the inputs with the switches on the basys3 and the outputs with the LEDs with the 'set_property' function. Then we integrated our code into basys3 with the 'generate bitstream' command. The coded basys3 is now able to react to the changes we make with the switches and LEDs, and we can test our circuit.

RESULTS:

Before starting our experiment, we created a truth table (Table 1) to observe the expected results.

Sunlight	Water	Poison	Aphid	Health (out)
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Table 1 : Truth Table

After creating the truth table, we can start writing our code (code1) with the methods we have. We took a look at the schematic of the code we wrote in the 'elaborated design' section on Vivado. As seen in Figure 1, we got the result we expected and completed this stage.

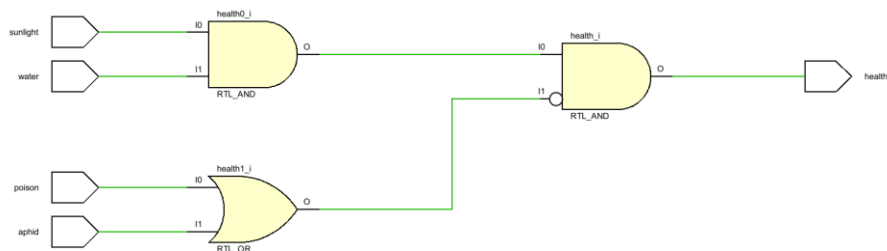


Figure 1 : The design scheme

We wrote another code (code2) to test and simulate the design we have, as stated in the methodology section. When we ran the simulation, we obtained the image shown in (Figure 2). The simulation we observed does not contradict the truth table we created.

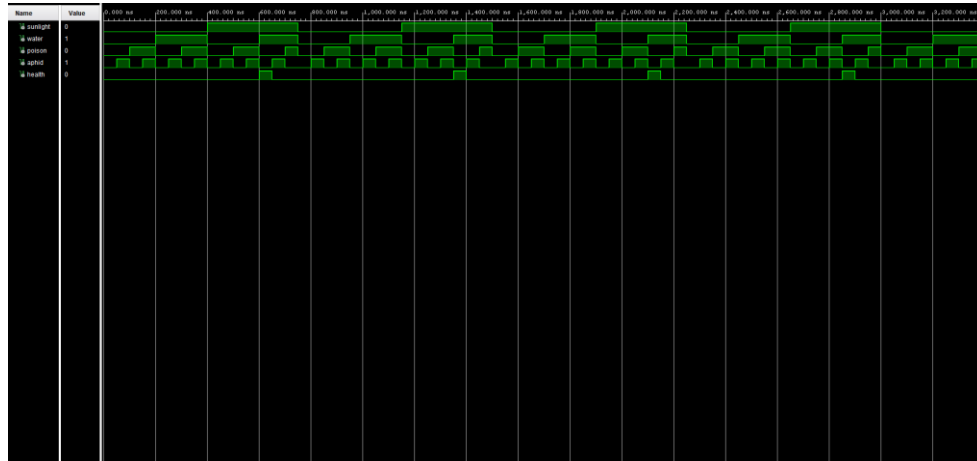


Figure 2 : Simulation

Now that we have successfully created the simulation, we will move on to apply the design we have taken as an example from real-life to the Basys3 FPGA board. We wrote a constraint file so that the switches and LEDs could work in relation to our design (code3).

Sunlight	:	R2 switch
Water	:	T1 switch
Poison	:	U1 switch
Aphid	:	W2 switch
Health	:	U16 LED

Combination 1 (Figure 3.0)

Sunlight : 1

Water : 1

Poison : 1

Aphid : 1

Expected health status : 0

Experimental health status : 0

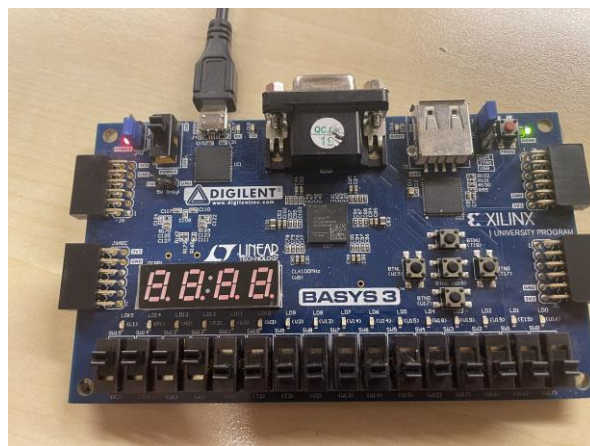


Figure 3.0

Combination 2 (Figure 3.1)

Sunlight : 0

Water : 0

Poison : 0

Aphid : 0

Expected health status : 0

Experimental health status : 0

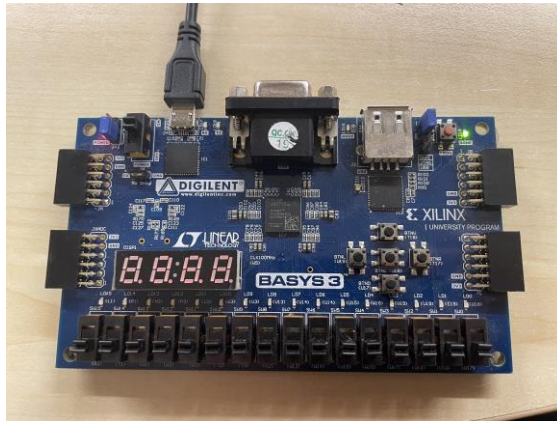


Figure 3.1

Combination 3 (Figure 3.2)

Sunlight : 1
 Water : 1
 Poison : 0
 Aphid : 0
 Expected health status : 1
 Experimental health status : 1

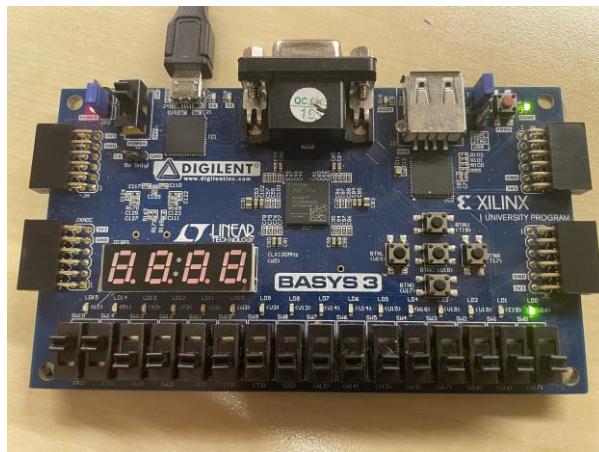


Figure 3.2

Combination 4 (Figure 3.3)

Sunlight : 1

Water : 1

Poison : 1

Aphid : 0

Expected health status : 0

Experimental health status : 0

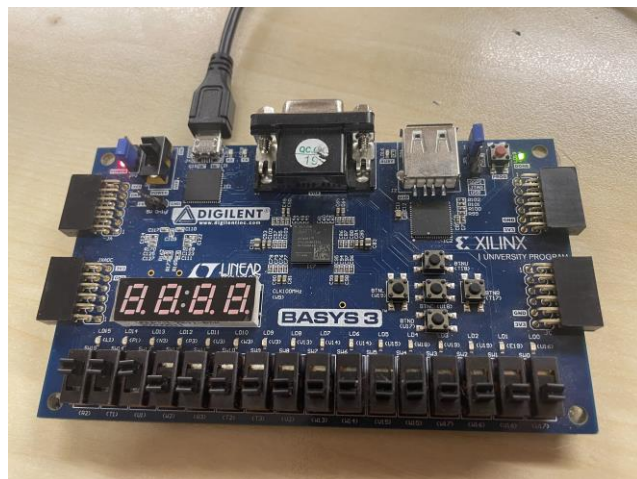


Figure 3.3

Combination 5 (Figure 3.4)

Sunlight : 1

Water : 0

Poison : 0

Aphid : 0

Expected health status : 0

Experimental health status : 0

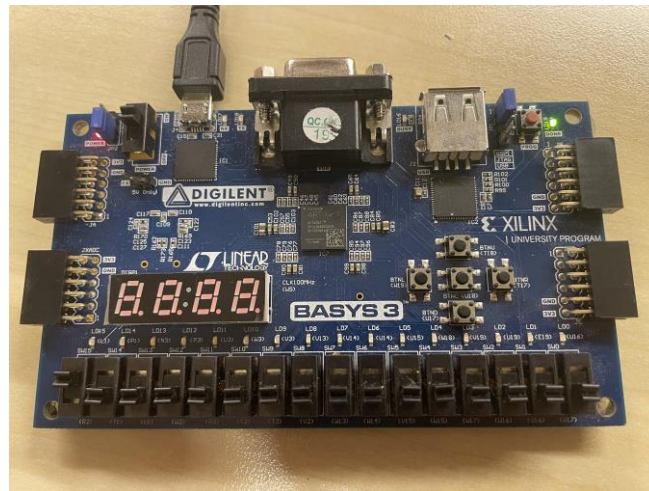


Figure 3.4

Combination 6 (Figure 3.5)

Sunlight : 1
 Water : 1
 Poison : 0
 Aphid : 1
 Expected health status : 0
 Experimental health status : 0

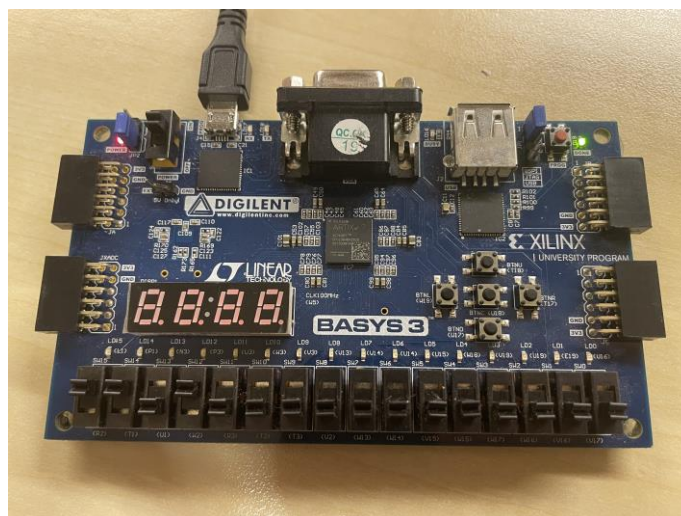


Figure 3.5

As can be seen, our observations were matching the expected outcomes.

CONCLUSION:

Our goal in this experiment was to learn how to code with VHDL and then understand how the VHDL code works on the Basys3 FPGA. We successfully implemented a real-life scenario which showing the health status of a pepper plant to our Basys3 FPGA. We created a truth table to list the theoretical results at the beginning, and we saw that the results in the table we created successfully overlapped with the results we observed on Basys3 in the experiment. We designed our combinational circuit using the logic designs, gate schematics and its rules ,and also truth tables we learned in the EE-102 course. Even if we encountered a few minor warnings while creating the codes, we managed to solve the problems through research and trial and error.

APPENDICES:

CODE1 (design)

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Buyuksarikulak_Furkan_22002097_EE102_Lab2 is
```

```
    Port ( sunlight : in STD_LOGIC;
```

```
          water : in STD_LOGIC;
```

```
          poison : in STD_LOGIC;
```

```
          aphid : in STD_LOGIC;
```

```
        health : out STD_LOGIC);  
end Buyuksarikulak_Furkan_22002097_EE102_Lab2;
```

```
architecture Behavioral of Buyuksarikulak_Furkan_22002097_EE102_Lab2 is  
begin  
--This is where we designed our gate combination  
health <= (sunlight and water) and (poison nor aphid);  
end Behavioral;
```

CODE2 (testbench)

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity testbench_furkan is  
end testbench_furkan;  
  
architecture Behavioral of testbench_furkan is  
COMPONENT Buyuksarikulak_Furkan_22002097_EE102_Lab2  
    Port ( sunlight : in STD_LOGIC;  
          water : in STD_LOGIC;  
          poison : in STD_LOGIC;  
          aphid : in STD_LOGIC;  
          health : out STD_LOGIC);  
END COMPONENT;  
  
SIGNAL sunlight : STD_LOGIC;  
SIGNAL water : STD_LOGIC;  
SIGNAL poison : STD_LOGIC;
```

```
SIGNAL aphid : STD_LOGIC;
```

```
SIGNAL health : STD_LOGIC;
```

```
BEGIN
```

```
UUT:Buyuksarikulak_Furkan_22002097_EE102_Lab2 Port Map (
```

```
sunlight => sunlight,
```

```
water => water,
```

```
poison => poison,
```

```
aphid => aphid,
```

```
health => health);
```

```
testbench_furkan : PROCESS
```

```
    BEGIN
```

```
        sunlight<='0';
```

```
        water<='0';
```

```
        poison<='0';
```

```
        aphid<='0';
```

```
        wait for 50 ns;
```

```
        sunlight<='0';
```

```
        water<='0';
```

```
        poison<='0';
```

```
        aphid<='1';
```

```
        wait for 50 ns;
```

```
        sunlight<='0';
```

```
water<='0';  
poison<='1';  
aphid<='0';
```

```
wait for 50 ns;  
sunlight<='0';  
water<='0';  
poison<='1';  
aphid<='1';
```

```
wait for 50 ns;  
sunlight<='0';  
water<='1';  
poison<='0';  
aphid<='0';
```

```
wait for 50 ns;  
sunlight<='0';  
water<='1';  
poison<='0';  
aphid<='1';
```

```
wait for 50 ns;  
sunlight<='0';  
water<='1';  
poison<='1';
```

aphid<='0';

wait for 50 ns;

sunlight<='0';

water<='1';

poison<='1';

aphid<='1';

wait for 50 ns;

sunlight<='1';

water<='0';

poison<='0';

aphid<='0';

wait for 50 ns;

sunlight<='1';

water<='0';

poison<='0';

aphid<='1';

wait for 50 ns;

sunlight<='1';

water<='0';

poison<='1';

aphid<='0';

wait for 50 ns;
sunlight<='1';
water<='0';
poison<='1';
aphid<='1';

wait for 50 ns;
sunlight<='1';
water<='1';
poison<='0';
aphid<='0';

wait for 50 ns;
sunlight<='1';
water<='1';
poison<='0';
aphid<='1';

wait for 50 ns;
sunlight<='1';
water<='1';
poison<='1';
aphid<='0';

wait for 50 ns;
sunlight<='1';

```
water<='1';  
poison<='1';  
aphid<='1';  
END PROCESS;  
end Behavioral;
```

CODE3 (constraints)

```
set_property PACKAGE_PIN R2 [get_ports {sunlight}]  
    set_property IOSTANDARD LVCMOS33 [get_ports {sunlight}]  
set_property PACKAGE_PIN T1 [get_ports {water}]  
    set_property IOSTANDARD LVCMOS33 [get_ports {water}]  
set_property PACKAGE_PIN U1 [get_ports {poison}]  
    set_property IOSTANDARD LVCMOS33 [get_ports {poison}]  
set_property PACKAGE_PIN W2 [get_ports {aphid}]  
    set_property IOSTANDARD LVCMOS33 [get_ports {aphid}]  
set_property PACKAGE_PIN U16 [get_ports {health}]  
    set_property IOSTANDARD LVCMOS33 [get_ports {health}]
```